

Tolerancia a Fallos en el Microservicio de Notificaciones

La tolerancia a fallos es uno de los principios más importantes dentro de una arquitectura basada en microservicios, ya que permite que un sistema continúe operando aun cuando alguno de sus componentes presente una interrupción temporal. En aplicaciones destinadas a la gestión de emergencias, como el proyecto **C5 Alert System**, resulta indispensable garantizar que ninguna alerta generada por los dispositivos ESP32 sea descartada, debido a que cada una representa un posible incidente que requiere atención inmediata.

Para cumplir este objetivo, el sistema implementa un mecanismo de almacenamiento temporal mediante Redis, el cual funciona como una cola de mensajes entre el microservicio de Recepción y el microservicio de Notificaciones. Esta arquitectura desacoplada permite que ambos servicios trabajen de forma independiente, evitando que la caída de uno de ellos afecte el funcionamiento del resto del sistema. Como consecuencia, la plataforma mantiene una alta disponibilidad y asegura la entrega de las alertas una vez que el servicio vuelve a estar operativo.

Flujo de funcionamiento

El proceso de notificación inicia cuando un dispositivo ESP32 detecta una situación de emergencia y envía la información correspondiente al sistema mediante el protocolo MQTT. Posteriormente, el microservicio **Reception** recibe la alerta, valida su contenido y la convierte a formato JSON para facilitar su procesamiento por el resto de la arquitectura.

En lugar de enviar la información directamente al Dashboard, el microservicio de Recepción almacena inicialmente cada alerta en la cola **notification_queue** de Redis. Este paso representa una medida de seguridad importante, ya que garantiza que la información permanezca disponible incluso si el microservicio de Notificaciones deja de funcionar temporalmente.

Una vez almacenada la alerta, el microservicio **Notification** permanece escuchando continuamente la cola de Redis. Cuando detecta la existencia de un nuevo mensaje, lo recupera y lo envía mediante WebSocket a todos los operadores que se encuentran conectados al Dashboard, permitiendo que la alerta sea visualizada en tiempo real.

De manera simplificada, el flujo de procesamiento implementado en el proyecto puede representarse de la siguiente manera:



Este diseño sigue el principio de desacoplamiento entre servicios, permitiendo que la recepción de nuevas alertas continúe funcionando aun cuando el servicio encargado de las notificaciones presente una interrupción temporal.

Un fragmento del código implementado en el microservicio de Recepción muestra el momento en que la alerta es almacenada dentro de Redis antes de continuar con su procesamiento:

```
payload_str = json.dumps(alert) redis_client.lpush("notification_queue",  
payload_str)
```

La utilización de la instrucción **LPUSH** inserta la alerta al inicio de la cola, garantizando que quede almacenada antes de ser enviada al microservicio de Notificaciones.

Retención y recuperación de mensajes

El mecanismo de retención de mensajes constituye la base de la estrategia de tolerancia a fallos implementada en el proyecto. Cada alerta permanece almacenada dentro de Redis hasta que el microservicio de Notificaciones pueda procesarla correctamente.

Cuando dicho microservicio deja de funcionar debido a una falla, un reinicio o tareas de mantenimiento, el microservicio de Recepción continúa operando con normalidad y sigue almacenando las nuevas alertas dentro de la cola **notification_queue**. Esto permite que el sistema continúe aceptando información proveniente de los dispositivos ESP32 sin provocar pérdidas de datos.

Al reiniciarse el servicio de Notificaciones, el consumidor de Redis vuelve a ejecutarse automáticamente y comienza a recuperar cada uno de los mensajes pendientes. Para ello utiliza la operación **BRPOP**, la cual permanece esperando la llegada de nuevos mensajes hasta que exista información disponible para ser procesada.

El comportamiento puede observarse en el siguiente fragmento del proyecto:

```
item = await redis_client.brpop(  
    "notification_queue", timeout=2  
)
```

La función **BRPOP** permite consumir las alertas almacenadas en Redis conforme van estando disponibles, evitando consultas constantes sobre la base de datos y optimizando el rendimiento del sistema.

Gracias a esta estrategia, las alertas generadas durante una interrupción temporal permanecen almacenadas y son procesadas automáticamente una vez que el servicio vuelve a encontrarse disponible. Este comportamiento incrementa significativamente la disponibilidad del sistema y garantiza la continuidad del proceso de notificación.

Reintentos automáticos

Además del almacenamiento temporal de mensajes, el sistema incorpora un mecanismo de reintentos automáticos con el propósito de incrementar la confiabilidad durante la entrega de las alertas.

Cuando el microservicio de Notificaciones intenta distribuir una alerta y ocurre algún problema durante el envío, o bien no existen operadores conectados para recibirla, el sistema evita eliminar definitivamente el mensaje. En su lugar, la alerta vuelve a

insertarse dentro de la cola de Redis para que pueda ser procesada nuevamente cuando las condiciones del sistema sean adecuadas.

Este comportamiento se implementa mediante una nueva inserción de la alerta en la cola:

```
await redis_client.lpush(  
    "notification_queue",  
    alert_data  
)
```

Gracias a este mecanismo, el sistema garantiza que ninguna alerta sea descartada por fallas temporales de comunicación o desconexiones inesperadas. En consecuencia, las notificaciones permanecen disponibles hasta que puedan entregarse correctamente, incrementando la resiliencia de toda la arquitectura.

Prueba de tolerancia a fallos

Con el propósito de validar el funcionamiento del mecanismo de tolerancia a fallos, se realizó la prueba descrita dentro de la documentación oficial del proyecto. El objetivo consiste en comprobar que el sistema continúa conservando todas las alertas aun cuando el microservicio de Notificaciones deje de funcionar temporalmente.

El procedimiento inicia deteniendo el contenedor correspondiente al microservicio de Notificaciones mediante el siguiente comando:

```
docker stop notification
```

Una vez detenido el servicio, se generan nuevas alertas desde el dispositivo ESP32. Durante este periodo, las alertas no deben aparecer inmediatamente en el Dashboard del operador, debido a que el servicio encargado de enviarlas permanece fuera de funcionamiento. Sin embargo, el microservicio de Recepción continúa almacenando cada alerta dentro de la cola **notification_queue** de Redis.

En este momento deben observarse los registros del sistema, donde es posible verificar que las alertas continúan siendo encoladas correctamente. Esta evidencia demuestra que el sistema mantiene una tasa de pérdida de mensajes del **0 %**, ya que ninguna alerta es descartada durante la interrupción del servicio.

Posteriormente, el microservicio de Notificaciones vuelve a iniciarse mediante el comando:

```
docker start notification
```

Después del reinicio, el consumidor de Redis detecta automáticamente las alertas pendientes y comienza a procesarlas en el mismo orden en que fueron almacenadas. Como resultado, todas las alertas acumuladas durante la interrupción aparecen en el Dashboard del operador sin necesidad de reenviarlas desde el dispositivo ESP32.

Las evidencias que deben mostrarse durante la demostración son las siguientes:

- Logs de Docker donde se observe que el contenedor del microservicio de Notificaciones fue detenido.
- Registros que indiquen que las alertas continúan almacenándose dentro de Redis mientras el servicio permanece inactivo.
- Reinicio del contenedor mediante `docker start notification`.
- Aparición automática de todas las alertas pendientes en la interfaz del operador una vez recuperado el servicio.

Los resultados esperados coinciden con la documentación de pruebas del proyecto, donde se establece que la pérdida de mensajes debe ser del **0 %** y que todas las alertas almacenadas deben entregarse automáticamente después del reinicio del servicio.

Conclusión

La implementación del mecanismo de tolerancia a fallos permitió comprobar que la arquitectura desarrollada es capaz de mantener la integridad de la información incluso cuando el microservicio de Notificaciones deja de funcionar temporalmente. La utilización de Redis como cola de mensajes proporciona un almacenamiento temporal que evita la pérdida de alertas y permite recuperar automáticamente el procesamiento una vez que el servicio vuelve a estar disponible.

Las pruebas realizadas demostraron que las alertas continúan siendo recibidas por el microservicio de Recepción, permanecen almacenadas durante la interrupción y son entregadas correctamente después del reinicio del servicio, obteniendo una tasa de pérdida de mensajes del **0 %**. Estos resultados evidencian que la solución implementada cumple con los principios de disponibilidad, resiliencia y continuidad del servicio, características indispensables en sistemas destinados a la gestión de emergencias donde la información no puede perderse bajo ninguna circunstancia.